SENDBIRD AI AGENT · MESSENGER SDK

대화 목록 헤더 버튼 커스터마이징 가이드

목록 화면의 **닫기(X)** → **대화 화면으로 돌아가기**

채팅 화면의 **목록 이동(≡)** 아이콘 변경 · 숨김 · 비활성화

대상 기능 · 헤더 슬롯 교체 (ConversationHeaderLayout / ConversationListHeaderLayout)

기준 버전 · @sendbird/ai-agent-messenger-react v1.45.0

문서 개정 · 2026-11-09 (숨김 적용 시 깜빡임 검증 내용 추가)

목차

1. 요청 내용과 결론
2. SDK 헤더 구조
   - 2.1 두 개의 화면과 두 개의 헤더
   - 2.2 헤더 슬롯 교체 방식
   - 2.3 교체 가능한 슬롯 목록
3. 목록 헤더 — 닫기(X)를 뒤로가기로
   - 3.1 기본 동작
   - 3.2 돌아갈 대화를 정하는 API
   - 3.3 구현 코드
4. 채팅 헤더 — 목록 이동(≡) 아이콘 변경
   - 4.1 기본 동작
   - 4.2 구현 코드
5. 채팅 헤더 — 목록 이동(≡) 숨기기 · 비활성화
   - 5.1 설정으로는 끌 수 없습니다
   - 5.2 숨기기 (권장)
   - 5.3 비활성화
   - 5.4 CSS로 숨기면 안 되는 이유
   - 5.5 버튼이 잠깐 보였다 사라진다면
6. 메신저에 연결
7. 동작 확인 결과
8. 구현 시 유의 사항

1 요청 내용과 결론

> **요청** — ① 대화 목록 화면 상단 헤더의 `X` 버튼을 누르면 위젯이 닫히는데, 닫는 대신 **직전 채팅 화면으로 돌아가게** 하고 싶다. ② 채팅 화면에서 **목록으로 들어가는 ≡ 아이콘**도 다른 모양으로 바꾸고 싶다. ③ 경우에 따라 그 아이콘을 **숨기거나 비활성화**하고 싶다.

결론: 세 건 모두 커스텀 작업이 필요하며, 모두 구현 가능합니다. 동작이나 아이콘을 바꾸는 설정값은 SDK에 없습니다. 다만 SDK가 헤더를 **슬롯(slot) 구조**로 공개하고 있어서, 해당 자리의 컴포넌트를 통째로 교체하는 방식으로 구현합니다. SDK를 수정하거나 내부 구현에 의존할 필요는 없고, **웹(React) 코드만** 수정하면 됩니다.

설정으로 가능한가	아니오 — 관련 설정값이 없습니다.
필요한 작업	웹(React) 컴포넌트 추가 + 메신저에 슬롯 연결 2줄 (숨김·비활성은 넣을 컴포넌트만 교체)
네이티브 수정	불필요 (Android / iOS 모두 변경 없음)
사용하는 API	모두 SDK가 공개(export)하는 공식 API

2 SDK 헤더 구조

2.1 두 개의 화면과 두 개의 헤더

메신저 위젯 안에는 화면이 두 개 있고, 헤더도 각각 따로 있습니다. 두 헤더는 서로 다른 레이아웃 컴포넌트를 쓰기 때문에 **한쪽만 골라서 바꿀 수 있습니다.**

화면	헤더 컴포넌트	이번에 바꿀 버튼
대화(채팅) 화면	`ConversationHeaderLayout`	왼쪽 ≡ **메뉴 버튼** → 아이콘 변경 · 숨김
대화 목록 화면	`ConversationListHeaderLayout`	오른쪽 **X 닫기 버튼** → 뒤로가기로 교체

2.2 헤더 슬롯 교체 방식

SDK는 헤더의 각 자리를 이름 붙은 슬롯으로 공개하고 있습니다. 바꾸고 싶은 슬롯에 `component` 로 내 컴포넌트를 넘기면 그 자리만 교체됩니다.

```
// 목록 화면 헤더의 오른쪽 영역 교체
<ConversationListHeaderLayout.EndArea component={MyBackButton} />
```

```
// 채팅 화면 헤더의 메뉴 버튼 교체
<ConversationHeaderLayout.MenuButton component={MyMenuButton} />
```

교체했더라도 **원래의 기본 컴포넌트가 필요할 때는 다시 꺼내 쓸 수 있습니다.** 아래 경로로 SDK 기본 구현에 그대로 접근합니다.

```
ConversationListHeaderLayout.defaults.components.CloseButton
ConversationHeaderLayout.defaults.components.MenuButton
```

2.3 교체 가능한 슬롯 목록

참고로 `ConversationHeaderLayout`(채팅 화면 헤더)이 공개하는 슬롯은 다음과 같습니다.

슬롯	자리
`StartArea`	헤더 왼쪽 영역 전체
`MenuButton`	대화 목록으로 가는 ≡ 버튼
`TitleArea` / `Title`	가운데 제목 영역
`EndArea`	헤더 오른쪽 영역 전체
`HandoffButton`	상담사 연결 버튼
`ConversationCloseButton`	대화 종료 버튼
`ExpandButton`	메신저 확장 버튼
`CloseButton`	위젯 닫기 버튼
`MemoryIndicator`	메모리 표시

영역 전체 vs 버튼 하나

`EndArea` 처럼 영역을 통째로 교체하면 그 안의 기본 버튼들이 모두 사라집니다. 버튼 하나만 바꾸고 싶다면 `MenuButton` 같은 **개별 슬롯**을 쓰는 편이 안전합니다.

3 목록 헤더 — 닫기(X)를 뒤로가기로

3.1 기본 동작

SDK의 목록 헤더 닫기 버튼은 **위젯 열림 상태를 false 로 만드는** 동작만 합니다. 화면 이동(navigation)과는 무관하므로, 눌렀을 때 위젯 자체가 사라집니다.

```
// SDK 내부 — ConversationListHeaderLayout 의 기본 CloseButton
const CloseButton = () => {
  const { state } = useMessengerContext();
  return <IconButton type="close" onClick={() => state.setOpened(false)} />;
};
```

참고

목록 화면으로 들어가는 경로는 헤더 왼쪽의 **메뉴(≡)** 버튼입니다. 즉 사용자는 채팅 → 메뉴 → 목록 순서로 이동했는데, 목록에서 나가는 버튼이 뒤로가 아니라 닫기로 되어 있어 흐름이 끊긴다는 것이 이번 요청의 배경입니다.

3.2 돌아갈 대화를 정하는 API

API	용도
`useConversationListContext()` → `onOpenConversationView(url, status)`	지정한 대화의 채팅 화면으로 이동
`useConversationListContext()` → `listSource.channels`	현재 목록에 로드된 대화 배열
`useMessengerSessionContext()` → `activeChannel`	직전까지 보고 있던 대화

돌아갈 대화는 ① **직전에 보던 대화(`activeChannel`)** → ② **목록의 첫 번째 대화** 순으로 고르면 자연스럽습니다. 둘 다 없는 경우(대화 이력이 하나도 없는 신규 사용자)에는 돌아갈 곳이 없으므로 **SDK 기본 닫기 버튼을 그대로 보여주는** 편이 안전합니다. 화살표만 띄워 두면 눌러도 아무 일이 일어나지 않아 오히려 혼란스럽기 때문입니다.

3.3 구현 코드

`web/src/components/ConversationListBackButton.tsx`

```
import {
  ConversationListHeaderLayout,
  useConversationListContext,
  useMessengerSessionContext,
} from '@sendbird/ai-agent-messenger-react';

export function ConversationListBackButton() {
  const { onOpenConversationView, listSource } = useConversationListContext();
  const { activeChannel } = useMessengerSessionContext();

  // 돌아갈 대화 결정 — 직전에 보던 대화 우선, 없으면 목록의 첫 번째
  const first = listSource.channels[0];
  const target = activeChannel
    ? { url: activeChannel.url, status: activeChannel.status }
    : first
      ? { url: first.url,
```

```
          status: (first.conversation?.status === 'closed' ? 'closed' : 'open') as 'open' |
'closed' }
      : null;

  // 돌아갈 대화가 없으면 SDK 기본 동작(위젯 닫기)을 그대로 둡니다.
  if (!target) {
    return <ConversationListHeaderLayout.defaults.components.CloseButton />;
  }

  return (
    <button
      type="button"
      aria-label="대화 화면으로 돌아가기"
      onClick={() => onOpenConversationView(target.url, target.status)}
      style={{ display: 'flex', alignItems: 'center', justifyContent: 'center',
               width: 32, height: 32, padding: 0, border: 'none',
               background: 'transparent', cursor: 'pointer', color: 'inherit' }}
    >
      {/* 닫기(X) 대신 '뒤로' 의미가 드러나도록 화살표를 씁니다. */}
      <svg width="22" height="22" viewBox="0 0 24 24" fill="none"
           stroke="currentColor" strokeWidth="2"
           strokeLinecap="round" strokeLinejoin="round">
        <polyline points="15 18 9 12 15 6" />
      </svg>
    </button>
  );
}
```

4 채팅 헤더 — 목록 이동(≡) 아이콘 변경

4.1 기본 동작

채팅 화면 헤더 왼쪽의 ≡ 버튼은 `useConversationContext()` 가 제공하는 `onNavigateToConversationList()` 를 호출해 목록 화면으로 이동합니다. 이 함수가 없는 구성(목록 기능을 쓰지 않는 경우)에서는 **버튼 자체를 렌더링하지 않습니다.**

API	용도
`useConversationContext()` → `onNavigateToConversationList()`	대화 목록 화면으로 이동

없을 수 있는 함수입니다

`onNavigateToConversationList` 는 **선택적(optional)** 값입니다. 커스텀 버튼에서도 `if (!onNavigateToConversationList) return null;` 처리를 반드시 넣어 SDK 기본 동작과 동일하게 맞춰 주세요. 빠뜨리면 목록 기능이 꺼진 구성에서 **눌러도 아무 일이 없는 버튼**이 노출됩니다.

4.2 구현 코드

`web/src/components/ConversationListMenuButton.tsx`

```
import { useConversationContext } from '@sendbird/ai-agent-messenger-react';

export function ConversationListMenuButton() {
  const { onNavigateToConversationList } = useConversationContext();

  // 목록 기능이 꺼진 구성에서는 SDK 기본 동작과 동일하게 아무것도 그리지 않습니다.
  if (!onNavigateToConversationList) return null;

  return (
    <button
      type="button"
      aria-label="대화 목록 열기"
      onClick={() => onNavigateToConversationList()}
      style={{ display: 'flex', alignItems: 'center', justifyContent: 'center',
               width: 32, height: 32, padding: 0, border: 'none',
               background: 'transparent', cursor: 'pointer', color: 'inherit' }}
    >
      {/* 햄버거(≡) 대신 '목록' 의미가 드러나는 아이콘을 씁니다. */}
      <svg width="22" height="22" viewBox="0 0 24 24" fill="none"
           stroke="currentColor" strokeWidth="2"
           strokeLinecap="round" strokeLinejoin="round">
        <line x1="9" y1="6" x2="20" y2="6" />
        <line x1="9" y1="12" x2="20" y2="12" />
        <line x1="9" y1="18" x2="20" y2="18" />
        <circle cx="4.5" cy="6" r="1.3" fill="currentColor" stroke="none" />
        <circle cx="4.5" cy="12" r="1.3" fill="currentColor" stroke="none" />
        <circle cx="4.5" cy="18" r="1.3" fill="currentColor" stroke="none" />
      </svg>
    </button>
  );
}
```

이미지 아이콘을 쓰고 싶다면

`<svg>` 자리에 `<img src="/icons/list.svg" width={22} height={22} alt="" />` 처럼 이미지를 넣어도 됩니다. 다만 이미지로 바꾸면 `color: 'inherit'` 에 의한 **테마 색 자동 추종이 동작하지 않으므로**, 다크 테마를 함께 쓴다면 테마별 이미지를 따로 준비해야 합니다.

5 채팅 헤더 — 목록 이동(≡) 숨기기 · 비활성화

5.1 설정으로는 끌 수 없습니다

먼저 확인해 둘 점이 있습니다. `FixedMessenger` 는 대화 화면에 **목록 이동 함수를 항상 주입합니다.** 번들을 확인해 보면 조건 분기 없이 무조건 전달됩니다.

```
// SDK 내부 — FixedMessenger 의 화면 구성
<Conversation
  onNavigateToConversationList={() => { ... navigate(convList); }}   // 항상 주입
  ... />
```

SDK 기본 `MenuButton` 은 이 함수가 **없을 때만** 버튼을 그리지 않습니다. 그런데 `FixedMessenger` 에서는 그 조건이 성립하지 않으므로, ≡ **버튼을 끄는 설정값은 존재하지 않습니다.**

```
// SDK 기본 MenuButton — 함수가 없을 때만 숨겨짐
const MenuButton = () => {
  const { onNavigateToConversationList } = useConversationContext();
  if (!onNavigateToConversationList) return null;   // FixedMessenger 에서는 해당 없음
  ...
};
```

참고

대시보드의 `다중 대화 허용` 설정(`is_multiple_active_conversations_enabled`)은 **이 버튼과 무관합니다.** 그 값은 "대화 시작" 시 새 대화를 만들지 기존 대화를 이어갈지를 정할 뿐이고, 헤더 버튼 노출에는 관여하지 않습니다.

따라서 **4장과 같은 슬롯 교체 방식**으로 처리합니다. 바꿔 넣을 컴포넌트만 다릅니다.

5.2 숨기기 (권장)

슬롯에 `null` 을 반환하는 컴포넌트를 넣으면 버튼 자리가 사라집니다.

```
const Empty = () => null;

<FixedMessenger appId={APP_ID} aiAgentId={AI_AGENT_ID} ...>
  <FixedMessenger.ConversationChildren>
    <ConversationHeaderLayout.MenuButton component={Empty} />
  </FixedMessenger.ConversationChildren>
</FixedMessenger>
```

이것이 전부입니다. 헤더의 나머지 버튼(최소화 · 확장 · 닫기)은 그대로 유지되고, 왼쪽 영역만 비워집니다.

컴포넌트를 JSX 안에서 만들지 마세요

`component={() => null}` 처럼 인라인으로 쓰면 렌더할 때마다 **새 함수가 되어 슬롯이 매번 다시 등록됩니다.** 위 예시처럼 `Empty` 를 컴포넌트 바깥에 **한 번만** 선언해서 넘기세요.

두 곳을 혼동하지 마세요

반드시 `FixedMessenger.ConversationChildren` (채팅 화면) 안에 넣어야 합니다.
`ConversationListChildren` (목록 화면)에 넣으면 적용되지 않습니다.

5.3 비활성화

버튼은 보이되 누를 수 없게 하려면 `disabled` 버튼을 넣습니다. 헤더 레이아웃을 그대로 유지해야 할 때만 쓰세요.

```
export function DisabledMenuButton() {
  return (
    <button
      type="button"
      disabled
      aria-label="대화 목록 열기 (사용할 수 없음)"
      style={{ display: 'flex', alignItems: 'center', justifyContent: 'center',
               width: 32, height: 32, padding: 0, border: 'none',
               background: 'transparent', color: 'inherit',
               opacity: 0.35, cursor: 'not-allowed' }}
    >
      <svg width="22" height="22" viewBox="0 0 24 24" fill="none"
           stroke="currentColor" strokeWidth="2"
           strokeLinecap="round" strokeLinejoin="round">
        <line x1="4" y1="6" x2="20" y2="6" />
        <line x1="4" y1="12" x2="20" y2="12" />
        <line x1="4" y1="18" x2="20" y2="18" />
      </svg>
    </button>
  );
}
```

특별한 이유가 없다면 숨기기를 권합니다

눌러도 아무 일이 없는 버튼은 사용자에게 **고장으로 읽힙니다.** 비활성 상태가 일시적이고 곧 다시 열릴 예정일 때만 쓰시고, 영구적으로 막을 것이라면 5.2 의 숨기기를 쓰세요.

비활성으로 두실 경우 `aria-label` 에 **사용할 수 없다는 사실을 함께** 적어 주세요. `disabled` 속성만으로는 스크린 리더 사용자가 "왜" 막혔는지 알 수 없습니다.

5.4 CSS로 숨기면 안 되는 이유

`display: none` 으로 가리는 방법이 떠오를 수 있지만 권장하지 않습니다. 이 버튼에는 **SDK가 보장하는 고정 id가 없어서**, 선택자를 만들려면 아래 둘 중 하나에 의존해야 합니다.

선택 방법	문제
클래스명 (`.sc-xxxx`)	styled-components 해시라 **빌드마다 바뀝니다.**
`[aria-label="메뉴"]`	레이블은 **언어 설정에 따라 바뀝니다.** 다국어 환경에서 깨집니다.

둘 다 SDK 업데이트나 언어 변경에 쉽게 깨집니다. 슬롯 교체는 **SDK가 공개한 공식 경로**이므로 이런 위험이 없습니다.

깜빡임의 원인이 되기도 합니다

CSS 로 숨기면 스타일이 적용되기 전 **한 프레임이 그대로 보이고**, 그 뒤 버튼 폭이 접히면서 **헤더 영역이 줄어드는 움직임**이 눈에 띕니다. `useEffect` 안에서 `querySelector` 로 DOM 을 직접 제거하는 방식도 같은 문제가 있습니다 (페인트 이후에 실행되기 때문입니다). 자세한 내용은 5.5 를 참고하세요.

숨기기 전에 확인하세요

≡ 버튼은 **대화 목록으로 들어가는 유일한 경로**입니다. 이 버튼을 숨기면 사용자는 목록 화면에 접근할 수 없게 되고, **3장의 목록 헤더 뒤로가기 버튼도 쓰일 일이 없어집니다.**

목록 자체는 쓰되 진입 경로만 바꾸고 싶다면, 버튼을 숨기는 대신 `entryPoint` 를 `'ConversationList'` 로 두어 **목록을 첫 화면으로** 띄우는 방법도 있습니다.

5.5 버튼이 잠깐 보였다 사라진다면

"채팅창이 열릴 때 ≡ 버튼이 한 번 보였다가 사라진다"는 문의가 있었습니다. 먼저 **구조상 어떤 일이 일어나는지** 정리합니다.

SDK 기본 `MenuButton` 이 먼저 렌더되고, 그 다음 슬롯이 교체합니다. 즉 **DOM 에는 기본 ≡ 가 잠깐 들어왔다가 빠지는 것이 맞습니다.** 이 순서 자체는 피할 수 없습니다.

다만 SDK 는 이 교체를 `useLayoutEffect` 에서 수행합니다. **브라우저가 화면을 그리기 직전에 교체가 끝나므로, 기본 ≡ 는 화면에 그려지지 않습니다.**

```
// SDK 슬롯 등록부 — v1.42.0 / v1.45.0 동일
s[slotName] = function ({ component }) {
  const { updateComponent } = useContext(ctx);
  useLayoutEffect(() => { component && updateComponent(slotName, component); }, [component]);
  return null;
};
```

실제로 프레임 단위로 측정한 결과입니다. 5.2 를 적용한 상태입니다.

측정 환경	기본 ≡ 가 화면에 그려진 프레임
Chrome — 데스크톱 레이아웃	0 / 358
Chrome — 모바일 레이아웃	0 / 720
iOS 27 WKWebView (웹뷰 진입과 동시에 채팅창 열림)	**0 / 349**

5.2 방식 자체는 깜빡이지 않습니다

세 환경 모두 기본 ≡ 가 그려진 프레임이 **0** 이었습니다. DOM 교체는 일어나지만(측정 로그상 `removed(메뉴)` 발생), 페인트 전에 끝납니다.

그래도 눈에 보인다면 아래를 순서대로 확인해 주세요.

확인할 것	왜
CSS 로도 함께 숨기고 있지 않은지	스타일 적용 전 한 프레임이 보이고 **폭이 접히며 영역이 줄어듭니다.**
`useEffect` 로 DOM 을 직접 제거하지 않는지	페인트 **이후**에 실행되므로 반드시 한 번 보입니다.
슬롯을 조건부로 렌더하지 않는지	`{ready && <...MenuButton ... />}` 형태면 `ready` 전까지 기본 버튼이 보입니다.
`component` 에 인라인 함수를 넘기지 않는지	렌더마다 새 함수가 되어 슬롯이 반복 등록됩니다. (5.2 주의 사항)
슬롯이 `ConversationChildren` 안에 있는지	위치가 다르면 교체 자체가 적용되지 않습니다.

직접 측정하는 방법

아래 스크립트를 페이지 `<head>` 맨 위에, **앱 번들보다 먼저** 넣고 채팅창을 열어 보세요.

```
<script>
  window.__frames = []; var t0 = performance.now();
  (function tick() {
    var w = document.getElementById('sb-agent-window'), menu = null;
    if (w) {
      var bs = w.querySelectorAll('button');
      for (var i = 0; i < bs.length; i++) {
        var l = bs[i].getAttribute('aria-label') || '';
        if (l.indexOf('메뉴') >= 0) { menu = l; break; }
      }
    }
    window.__frames.push({ vis: w ? getComputedStyle(w).visibility : null, menu: menu });
    if (performance.now() - t0 < 8000) requestAnimationFrame(tick);
  })();
</script>
```

채팅창을 연 뒤 콘솔에서 확인합니다.

```
window.__frames.filter(f => f.vis === 'visible' && f.menu).length
```

결과	해석
`0`	화면에는 그려지지 않았습니다. 보이는 것은 다른 원인입니다.
`1` 이상	실제로 그려졌습니다. 위 점검 목록을 확인하세요.

확실한 회피책 — 미리 마운트하기

메신저를 **채팅창을 열 때 마운트하지 말고**, 미리 마운트해 두고 **노출 여부만 토글**하세요. 교체가 **아직 보이지 않는 동안** 끝납니다.

```
// 권장 — 항상 마운트해 두고 opened 로 노출만 제어
const [opened, setOpened] = useState(false);

<FixedMessenger state={{ opened, setOpened }} ...>
  <FixedMessenger.ConversationChildren>
    <ConversationHeaderLayout.MenuButton component={Empty} />
  </FixedMessenger.ConversationChildren>
</FixedMessenger>
```

측정으로 확인한 차이

미리 마운트한 구성에서는 교체 시점의 창 상태가 `visibility: hidden` 이었습니다. 반대로 **마운트와 동시에 여는** 구성(`useState(true)`)에서는 `visible` 상태에서 교체가 일어납니다. (앞의 측정에서는 두 경우 모두 그려진 프레임이 0 이었지만, 기기나 네트워크가 느려 렌더가 여러 단계로 나뉘면 후자가 불리합니다.)

웹뷰 진입과 동시에 띄워야 한다면, 마운트와 노출 사이를 **한 틱만 벌려도** 같은 효과를 얻습니다.

6 메신저에 연결

`web/src/pages/SimpleChatPage.tsx`

```
<FixedMessenger ...>
  {/* 채팅 화면 헤더 — 아래 셋 중 하나를 넣습니다.
      ConversationListMenuButton : 아이콘만 교체  (4장)
      Empty / HiddenMenuButton   : 버튼 숨김      (5.2)
      DisabledMenuButton         : 보이되 비활성  (5.3)  */}
  <FixedMessenger.ConversationChildren>
    <ConversationHeaderLayout.MenuButton component={ConversationListMenuButton} />
  </FixedMessenger.ConversationChildren>

  {/* 대화 목록 화면 헤더 */}
  <FixedMessenger.ConversationListChildren>
    <ConversationListHeaderLayout.EndArea component={ConversationListBackButton} />
  </FixedMessenger.ConversationListChildren>
</FixedMessenger>
```

슬롯을 넣는 위치에 주의하세요

두 블록을 바꿔 넣으면 적용되지 않습니다.

· 채팅 화면 헤더 → `FixedMessenger.ConversationChildren`

· 목록 화면 헤더 → `FixedMessenger.ConversationListChildren`

7 동작 확인 결과

샘플 앱에 위 코드를 적용하고 실제로 확인한 결과입니다.

확인 항목	결과
채팅 화면 헤더 왼쪽	≡ 대신 목록 아이콘 표시
목록 아이콘 클릭 시	대화 목록 화면으로 정상 이동
목록 화면 헤더 오른쪽	`X` 대신 `<` (뒤로) 아이콘 표시
`<` 클릭 시	위젯이 닫히지 않고 **직전 채팅 화면으로 복귀**
복귀 후 상태	기존 대화 내용·입력창 그대로 유지
채팅 헤더의 나머지 버튼	변화 없음 — 기존 버튼 그대로 유지
`Empty` 적용 (5.2)	헤더에서 목록 버튼이 **사라짐** (나머지 버튼 유지)
`DisabledMenuButton` 적용 (5.3)	버튼은 보이고 `disabled` 상태, 클릭해도 **화면 이동 없음**
숨김 적용 시 깜빡임 (5.5)	iOS 27 WKWebView 포함 3개 환경에서 **그려진 프레임 0**

확인 완료

채팅 ↔ 목록 양방향 이동이 의도대로 작동하며, 이동 중 위젯 열림 상태가 유지되는 것까지 확인했습니다.

8 구현 시 유의 사항

대화 이력이 없는 경우

신규 사용자처럼 대화가 하나도 없으면 돌아갈 대상이 없습니다. 위 코드는 이때 **SDK 기본 닫기 버튼으로 자동 전환**되므로 별도 처리가 필요 없습니다.

목록 필터 설정의 영향

진행 중인 대화만 보이도록 목록 필터(`conversationStatusFilter`)를 걸어 둔 경우, 종료된 대화는 `listSource.channels` 에 포함되지 않습니다. 위 코드는 `activeChannel` 을 먼저 보기 때문에 **종료된 대화를 보던 중이라도 그 대화로 정상 복귀합니다.**

아이콘 디자인

예시는 외부 의존성 없이 인라인 SVG로 아이콘을 그렸습니다. 서비스에서 쓰는 아이콘 세트가 있다면 `<svg>` 부분만 교체하면 됩니다. 버튼 색상은 `color: 'inherit'` 로 두어 헤더 테마 색을 자동으로 따라갑니다.

터치 영역

모바일 웹뷰에서는 버튼 크기를 **최소 32×32px 이상**으로 유지해 주세요. 아이콘 자체를 작게 그리더라도 `<button>` 의 크기는 줄이지 않는 편이 오터치를 줄이는 데 유리합니다.

접근성

`aria-label` 을 반드시 남겨 주세요. 아이콘만 있는 버튼은 스크린 리더가 읽을 텍스트가 없어 레이블이 빠지면 시각 보조 기술 사용자가 용도를 알 수 없습니다.

본 문서는 @sendbird/ai-agent-messenger-react v1.45.0 기준으로 작성되었습니다. 5.5 의 측정값은 v1.45.0, iOS 27 WKWebView / Chrome 기준이며, 슬롯 등록 구현은 v1.42.0 과 v1.45.0 이 동일함을 확인했습니다. 사용된 API는 모두 SDK가 공개적으로 export 하는 항목이며, 내부 구현에 의존하지 않습니다. SDK 버전 업그레이드 시 동작을 재확인하시기 바랍니다.